

学号：22920202200773

姓名：孟媛

实验 11 龙贝格求积

一、实验目的

使用龙贝格求积方式进行数值积分。

二、数值计算原理

可以证明，用 T_N 和 T_{2N} 进行合理的线性组合可以得到一个比 T_N 和 T_{2N} 都好的近似求积公式，因此可以用 $\frac{4T_{2N} - T_N}{4 - 1}$ 作为 $I = \int_a^b f(x)dx$ 的近似计算方式，通过验算，可以得到 $S_N = \frac{4T_{2N} - T_N}{4 - 1}$ 即复化 Simpson 公式。复化 Simpson 公式要优于复化梯形公式。

同样，也可以证明，用 S_N 和 S_{2N} 进行合理的线性组合可以得到一个比 S_N 和 S_{2N} 都好的近似公式。通过验算，上式右端项是 C_N 。即 $C_N = \frac{4^2 S_{2N} - S_N}{4^2 - 1}$ 这就是复化 Cotes 公式，其要优于复化 Simpson 公式。

类似于前面的推导，可以得到关于 C_N 的线性组合公式 $I \approx \frac{4^3 C_{2N} - C_N}{4^3 - 1}$ ，令上式的右端项为 R_N 。即 $R_N = \frac{4^3 C_{2N} - C_N}{4^3 - 1}$ ，称 R_N 为龙贝格值。

可以猜想， R_N 要优于 Cotes 值 C_N 。因此用龙贝格值计算积分，将有较快的收敛速度。

这就是龙贝格求积方法。

三、源程序介绍

函数接受四个参数：积分下界，积分上界，精度（不小于 $1e-6$ ），被积函数。显示求积过程并返回积分结果。

函数计算过程：

- （1）控制精度不小于 $1e-6$ ，令初始步长为 $b-a$
- （2）准备初值，先用梯形公式计算积分近似值

- (3) 按变步长梯形公式计算积分近似值
- (4) 按加速公式依次求各积分
- (5) 当精度达到要求，则停止，否则步长折半，继续进行求积

```

1  import numpy
2  import scipy.integrate as si
3
4  #f函数定义
5  def f(x):
6      return numpy.sin(x) / x
7
8  #龙伯格算法
9  def Romberg(a, b, eps, f):
10     T = [] # 复化梯形序列
11     S = [] # Simpson序列
12     C = [] # Cotes序列
13     R = [] # Romberg序列
14     h = b - a
15     T.append(h * (f(a) + f(b)) / 2)
16     ep = eps + 1
17     m = 0
18     mul = 1
19     mul2 = 1
20     while(ep >= eps):
21         m = m + 1
22         mul = mul * 4
23         t = 0
24         for i in range(mul2 - 1):
25             t = t + f(a + (2 * (i + 1) - 1) * h / (mul2 * 2)) * h / (mul2 * 2)
26         t = t + T[-1] / 2
27         T.append(t)
28         if m >= 1:
29             S.append((mul * T[-1] - T[-2]) / (mul - 1))
30         if m >= 2:
31             C.append((mul * S[-1] - S[-2]) / (mul - 1))
32         if m >= 3:
33             R.append((mul * C[-1] - C[-2]) / (mul - 1))
34         if m > 4:
35             ep = numpy.fabs(10 * (R[-1] - R[-2]))
36             mul2 *= 2
37     return R
38
39 if __name__ == '__main__':
40     a = 1e-32
41     b = 1
42     eps = 1e-5
43     print('自编龙贝格求积函数结果为:', Romberg(a, b, eps, f)[-1])
44     area = si.quad(f, a, b)
45     print('api自带函数结果为:', si.quad(f, a, b)[0])
46

```

四、程序执行情况

4.1 书上给的例子:

```
#f函数定义
def f(x):
    return numpy.sin(x) / x
```

```
40 if __name__ == '__main__':
41     a = 1e-32
42     b = 1
43     eps = 1e-6
44     print('自编龙贝格求积函数结果为:', Romberg(a, b, eps, f)[-1])
45     area = si.quad(f, a, b)
46     print('api自带函数结果为:', si.quad(f, a, b)[0])
```

自编龙贝格求积函数结果为: 0.9460829816357201
api自带函数结果为: 0.9460830703671831

4.2 书上未给的例子

```
def f(x):
    return numpy.exp(x)
```

```
if __name__ == '__main__':
    a = 1e-32
    b = 1
    eps = 1e-5
    print('自编龙贝格求积函数结果为:', Romberg(a, b, eps, f)[-1])
    area = si.quad(f, a, b)
    print('api自带函数结果为:', si.quad(f, a, b)[0])
```

自编龙贝格求积函数结果为: 1.7182808099311995
api自带函数结果为: 1.7182818284590453

五、结果分析

通过对比可以发现，龙伯格求积法所得结果与自带函数求得的结果也极为接近，但是，它的计算过程十分繁琐，运行速度肉眼可见的非常慢。虽然可以得到较高精度的结果，但是就运行速度来看，在日常计算中性价比并不高。

六、实验体会

此次实验，我认识到了龙伯格求积法优秀的精度，但同时也认识到了它繁琐的计算过程以及缓慢的求积时间。总体来说，在日常计算精度要求不是很高的情况下，龙伯格求积法大

抵不是一种较优的选择,但在高精度要求的情况下,龙伯格求积法则是一种非常优秀的方法。